

Author's Note

Yu-Chi Chuang

Design Choices

I chose this topic because it directly relates to my current experience working with Unity gameplay systems. Rather than exploring AI at a conceptual or asset-generation level, I wanted to examine how AI-generated code behaves when placed inside an actual runtime system. The goal was to move beyond “AI can generate code” and instead evaluate whether that code produces correct gameplay behavior.

I intentionally selected a simple roll-a-ball prototype so that failures would be easy to observe and isolate. This allowed me to focus on specific design decisions, such as execution timing and color comparison, rather than dealing with large system complexity. I also chose to demonstrate two distinct failure modes: one related to physics execution (Update vs FixedUpdate) and one related to gameplay logic (color matching).

What I left out was more advanced AI integration, such as dynamic behavior or machine learning systems. These would introduce additional variables that could obscure the core argument. The purpose of this project is not to explore AI complexity, but to demonstrate that even simple AI-generated code can fail if it is not aligned with the system it operates in.

This essay reflects the course's master claim by showing that AI is a pipeline decision. The failure did not come from missing functionality, but from placing correct logic in the wrong context.

Tool Usage

Bookie the Bookmaker was used to generate initial drafts of the scenario and mechanism sections. The generated content helped structure the explanation but tended to remain at a high level. I revised the output to include concrete details about Unity's execution model, particularly the distinction between Update and FixedUpdate, to ensure the explanation accurately reflected the observed behavior.

Eddy the Editor was used to improve clarity and strengthen the causal chain in the argument. It identified sections where the explanation described behavior without clearly linking it to design decisions. Based on this feedback, I rewrote the failure case to explicitly connect incorrect implementation choices with runtime consequences.

Figure Architect was used to generate ideas for visual diagrams, such as comparing Update and FixedUpdate loops and illustrating the color-matching logic. While these figures were not fully included in the final submission, the prompts helped clarify which parts of the system required more precise explanation.

Overall, AI tools were helpful in structuring ideas, but they required human correction to align with the actual system behavior. This reinforces the central argument of the project: AI output must be interpreted and constrained, not accepted directly.

Self-Assessment

Argumentative Rigor (35 pts):

I believe this project performs strongly in this category. The essay presents a clear claim and supports it with two concrete failure cases. Each failure is traced from design decision to runtime behavior to gameplay consequence. The failures are also demonstrated in the implementation, not just described.

Technical Implementation (25 pts):

The Unity project includes both a working implementation and a reproducible failure case. The Human Decision Node is explicitly documented, and the differences between the AI-generated version and the revised version are visible. The system is simple but sufficient to demonstrate the core argument.

Clarity (20 pts):

The essay follows the required five-section structure and avoids unnecessary jargon. Each section builds on the previous one, and the explanation moves from intuition to mechanism to consequence.

Relative Quality (20 pts):

The project clearly demonstrates a failure mode that can be reproduced quickly. The Human Decision Node is explicit, and the argument aligns closely with the course's central claim. The main limitation is that the system is relatively simple, but this simplicity also makes the failure easier to understand.

Overall, I would evaluate this project as strong in demonstrating how AI-generated code can fail when not properly integrated into a runtime system.